Kosh

A fee ledger, parent portal and dues desk for Company A, Jaipur. Parents know what they owe without calling. Reminders send themselves and stop on payment. Accounts get one view of every centre.

GitHub	https://github.com/amukta14/kosh
Live app	https://kosh-alpha.vercel.app (Vercel + Turso, demo mode on)
Run locally	`npm install && npm run setup && npm run dev` (see deployment.md)
Demo logins	Parent `98290 00587` (code shown on screen) · Accounts `neha@companya.test` · Counsellor `vikram@companya.test` · password `kosh-demo-2026`

1. What I built, in one page

The brief's core slice, end to end, on realistic data: 184 students across 4 centres and 16 batches, with seven months of payments, reminders, attendance and test scores, including the three sample students exactly as given.

Core build item	Where it is
Fee ledger	Staff: Students → any student. Plan, discount, instalments, FIFO allocation, balance, days overdue, receipts, messages sent, record a payment.
Parent portal with dues and pay link	OTP login. First screen is amount, due date and one pay button. Fee plan and GST receipts, attendance and test scores. Hindi and English.
Reminders that stop on payment	Daily job, 7 steps per instalment (7 and 3 days before, on the day, 3, 7, 15, 30 after). Paying cancels the rest at once, because each run reads the live ledger.
Refund calculator	Policy written as code (clauses R1 to R7), the same clauses rendered on the public policy page. Shows its working line by line, the 10-day deadline, a GST credit note. Maker-checker approval.
Dues dashboard	Overdue by centre × batch heatmap, ageing buckets, the brief's 60-day metric against its baseline and target, collections due in the next four weeks, Tally XML and CSV export.
Roles	Parent, accounts executive, and counsellor (the brief says counsellors mark false flags, so they get their own role with only the call list).
AI: dropout-risk flag	Explainable logistic score from missed instalments, attendance and test trends. High risk opens a counsellor call task only. "Unsure" is shown but opens nothing. Counsellors mark false flags, which suppress re-flagging and feed a live precision figure.

Optional depth also built: bulk CSV upload of attendance and test scores per batch, payment matching with an unmatched queue, Tally export, DPDP consent capture, GST receipts, a test-mode payment gateway with signed webhooks, a ⌘K command palette, a WhatsApp preview of what the parent received,

and a **demo clock** that moves the business date forward and runs the daily jobs, so a reviewer can watch reminders fire and stop.

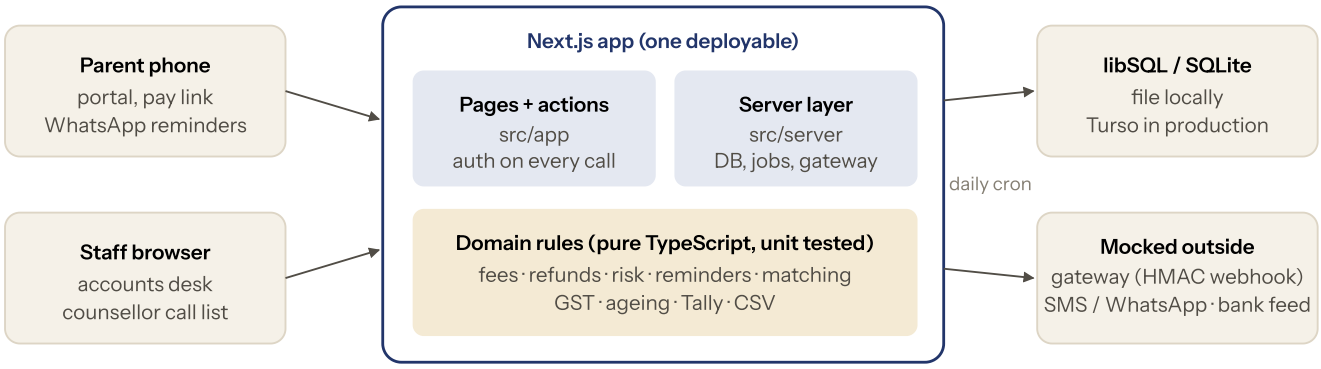
2. Assumptions

Everything below is my assumption, not something the brief states. Each is labelled so it can be checked with the client.

#	Assumption	Why
A1	Demo "today" is **2 Nov 2026**.	Only date where the sample rows agree: 10 Oct + 23 days overdue, and 15 Nov − 13 days.
A2	2027 batches run 6 Apr 2026 to 31 Mar 2027 (360 days).	Usual Kota/Jaipur session for a 2027 exam.
A3	Gross fee ₹1,20,000. JEE: 4 × ₹30,000 on 10 Apr, 10 Jul, 10 Oct, 10 Jan. NEET: 3 × ₹40,000 on 15 Apr, 14 Aug, 15 Nov.	Matches the brief's average fee, "3 or 4 instalments" and the sample amounts and dates.
A4	Fees are GST-inclusive at 18%, SAC 999293, intra-state so CGST 9% + SGST 9%.	Coaching is taxable at 18%; all centres are in Rajasthan.
A5	Part payments clear the oldest instalment first (FIFO).	How fee desks usually work; keeps "days overdue" about the oldest debt.
A6	Reminders go by WhatsApp (mocked), one per step, at the 08:30 IST daily run, in the parent's chosen language.	Parents use low-end phones and WhatsApp already (scores go out there today).
A7	Refund policy text R1 to R7 (see section 5) is my reading of the 2024 guidelines: pro-rata by day for the unattended period, unreturned printed material deductible at list price, no other charges, paid within 10 days of the written request.	The brief requires a published, pro-rata policy but does not give the client's wording.
A8	GSTIN, company legal name and Tally ledger names are placeholders.	Not in the brief. No real identifiers in the repo.
A9	An OTP-verified login from the mobile number the centre holds for the parent counts as verifiable parental consent under DPDP, recorded with time and policy version. Retention 8 years after the course (tax records).	DPDP rules allow verification against reliable identity details already held.
A10	Risk model coefficients are hand-set.	No history of who dropped out exists yet. Section 6 explains how it gets refit.
A11	A bank credit is applied automatically only if its narration contains a valid student ID and the amount is within that student's balance.	A wrong match would mark someone else's child as paid.
A12	Pay links open without a login (signed, unguessable, show only first name and amount).	One tap from WhatsApp on a basic phone; logging in first would cost payments.

A13	Refunds need two accounts people (maker-checker).	Money leaving the business; common control.
A14	Seed data is a 184-student sample, not 2,600. All names and numbers are fictional.	Enough to make every screen realistic; the code paths do not change with size.

3. Architecture



Why this shape

- **One Next.js app, server-rendered.** The parent screens ship about 105 kB of JavaScript and render on the server, which suits low-end phones on patchy data. Server actions remove a separate API layer, so a two-person team can run it. Six weeks and ₹6 lakh do not pay for a microservice estate.
- **The hard rules are pure functions.** Instalments, FIFO allocation, refunds, risk, reminder schedules, matching and GST take plain data and "today" as input. That makes them easy to test, easy to read in an interview, and the same code runs in the browser for the live refund preview and on the server when the refund is saved.
- **Balances are computed, not stored.** The ledger is derived from instalments plus payments every time. There is no running balance that can drift out of step with receipts, which is the root cause of month-end reconciliation taking four days.
- **SQLite via libSQL.** The whole school is a few thousand rows a year. A file locally, Turso when hosted, same driver and migrations. Drizzle gives typed queries and a migration file in the repo.
- **Everything outside is behind one seam.** The gateway, the SMS/WhatsApp sender and the bank feed are mocked, but each mock goes through the same path the real one would: the test checkout posts a signed callback to the same handler as `/api/gateway/webhook`; reminders are written to an outbox table that a sender would drain.

4. Data model

Table	Holds	Notes
centres, batches	4 centres, 16 batches (course, start, end)	Batch dates drive the refund period.

parents	Name, mobile, language, consent time and version	Unique mobile; language also sets reminder language.
students	ID (STU-0587), batch, parent, joined, status, withdrawn on	Several students can share a parent (siblings).
fee_plans	Gross fee, discount type/value/reason, net fee	Percent or flat discount, validated.
instalments	Student, sequence, due date, amount	Parts always sum to the net fee.
payments	Amount, date, method, reference, receipt number, gateway order, who recorded it	The only source of truth for money in. Unique receipt and gateway order.
gateway_orders	Order per pay attempt, fixed amount, status	Amount taken from the ledger on the server.
unmatched_payments	Credits the matcher would not place, reason, resolution	Nothing is marked paid on a guess.
reminders	Instalment, step, sent or superseded, language, body, balance at send	Unique (instalment, step): a step can never send twice.
attendance, test_scores	Monthly present/total; tests with score/max	Upserted by bulk upload.
risk_flags	Probability, factors (JSON), status open/called/false flag, note, reviewer	Only ever read by the call list.
refunds	Request and leave dates, deduction, amount, calculation snapshot, policy version, deadline, status, maker, checker, payout ref	Snapshot keeps old refunds explainable after a policy change.
staff, otp_codes, audit_log, settings	Logins, hashed OTPs, every money and consent action, receipt counters and demo clock	Audit rows commit inside the same transaction as the payment.

Money is stored as whole rupees. GST is split in paise on receipts and exports so CGST + SGST + taxable always add back exactly to the amount received.

5. Main flows

Parent pays from a reminder

1. 08:30 job sees instalment 3 is 15 days overdue, sends the +15 step in Hindi with a signed pay link.
2. Link opens with no login: first name, amount, due date, one button.
3. Server creates a gateway order for the ledger balance (never a client amount).
4. Gateway calls back signed. Handler checks HMAC, order, amount, and that it is not already paid, then records the payment, receipt number and audit row in one transaction.
5. Instalment balance is now zero, so the next job run sends nothing for it and the trail shows the remaining steps as stopped. The parent is told how many upcoming reminders were cancelled, and gets a GST receipt.

Reminder rules

- Steps at −7, −3, 0, +3, +7, +15, +30 days.
- If the job missed days (or the demo clock jumps a week), only the latest due step is sent; older ones are marked superseded. Parents never get a burst.
- Withdrawn students get nothing.
- The amount in each message is the live balance, so part payments are reflected.
- Idempotent: a unique index on (instalment, step) means re-running the job is always safe.

Bank credit arrives

- Narration has a valid student ID and amount ≤ balance: applied, receipt issued.
- Anything else (no ID, unknown ID, overpayment, gateway order mismatch) goes to the queue with a reason and name-based suggestions. A person applies it after a confirm dialog, or marks it returned.

Refund rules (the hard part)

Clause	Rule as implemented
R2	Period starts at the later of course start and joining date; days used run to the last day attended, inclusive, capped at the course length.
R3	Fee for days used = net fee (after discount) × days used ÷ total days, rounded to the rupee.
R4	Unreturned printed material at list price, capped at ₹15,000 (above that needs a manual review). No admission, processing or cancellation charge.
R5	Refund = paid − fee for days used − material. Never below zero; a shortfall is shown for the record but not demanded.
R6	Deadline = written request date + 10 days. Shown as a countdown, red at 3 days.
R7	GST credit note split for the refunded amount.

Worked example from the seed: Aarav paid ₹90,000 of ₹1,20,000. Leaving after 218 of 360 days, fee for days used is ₹72,667, refund ₹17,333, due within 10 days. Saving the refund marks the student withdrawn (reminders stop at once); a second accounts person approves it, then it is marked paid with the bank reference.

6. How the AI part works, and what happens when it is wrong

The dropout-risk flag is a small logistic model, chosen over a black box because a counsellor has to be able to see *why* before calling a parent about their child.

Signal	Feature	Weight
Missed instalments	Instalments unpaid more than 7 days	0.85 each
Age of debt	Days the oldest is overdue, per 30, capped at 90	0.6
Low attendance	Below 80% in the latest month	4.0 × gap
Falling attendance	Earlier months vs last two, percentage points	0.06 per pt
Falling scores	Least-squares slope over the last 3 or 4 tests	0.09 per pt

Probability = sigmoid(−2.9 + sum of contributions). Each contribution is stored with the flag and drawn as a bar on the call card.

Guard rails

- **It can only create a call task.** The risk job's only write is to `risk_flags`; it has no code path to fees, reminders or parent messages.
- **Unsure is a first-class answer.** Between 35% and 60% the student is listed under "Watching" with the reasons, but nobody is asked to call. A single signal on its own can never reach "high". With under two months of attendance and two tests, and no fee signal, the answer is "too little data".
- **Counsellors correct it.** Closing a task needs a note and an outcome: called, or false flag. A false flag suppresses that student for 30 days unless risk rises by 15 points. The call list shows live precision (real flags ÷ reviewed).
- **It learns later, not silently.** Reviewed flags are the labels needed to refit the weights once a term of outcomes exists. Weights only change in a reviewed release (model version is shown on the page).

Payment matching is the other automation. Its failure mode is handled the same way: when unsure it does nothing and asks a person (section 5).

7. UI and design decisions

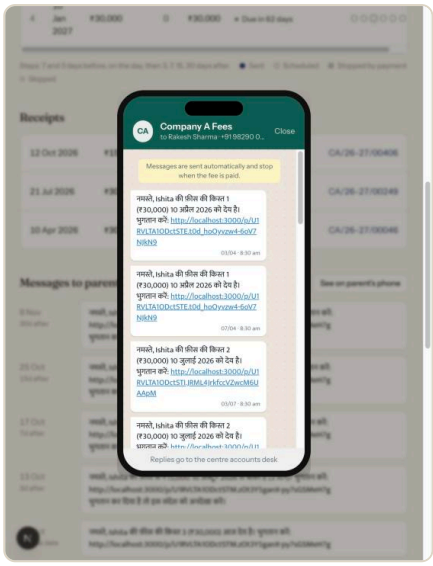
The client asked for restrained, purposeful design and for parents to understand their dues in five seconds. The palette comes from Jaipur block print (Sanganer, Bagru): indigo on unbleached cotton, with madder red, ochre and leaf green. Each colour has one job: `indigo = act` `madder = overdue` `ochre = part paid / watch` `leaf = paid`. Amounts are set in Fraunces, a serif with tabular figures, so numbers read like a ledger; interface text is Instrument Sans with Mukta for Devanagari. No gradients, no stock icons, no decorative charts.



Parent, first screen. Amount, due date with days late, one pay button, nothing else. Hindi by default for this parent; one tap switches to English and also changes reminder language.



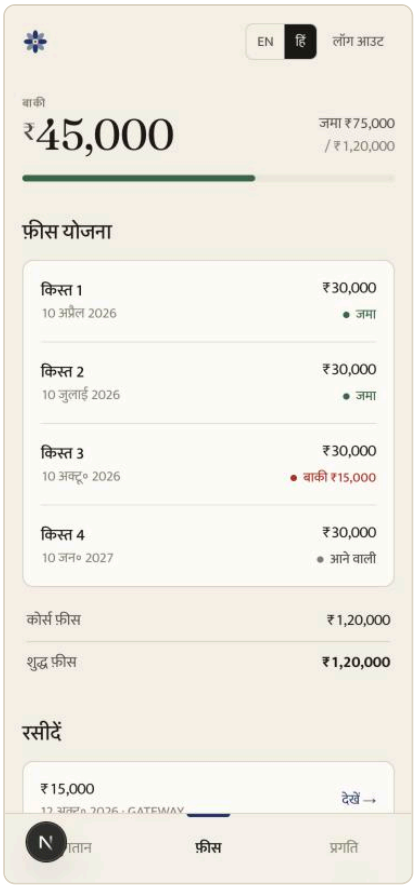
After paying. A stamped receipt number and a plain statement that the remaining reminders are cancelled, so the parent trusts the next message.



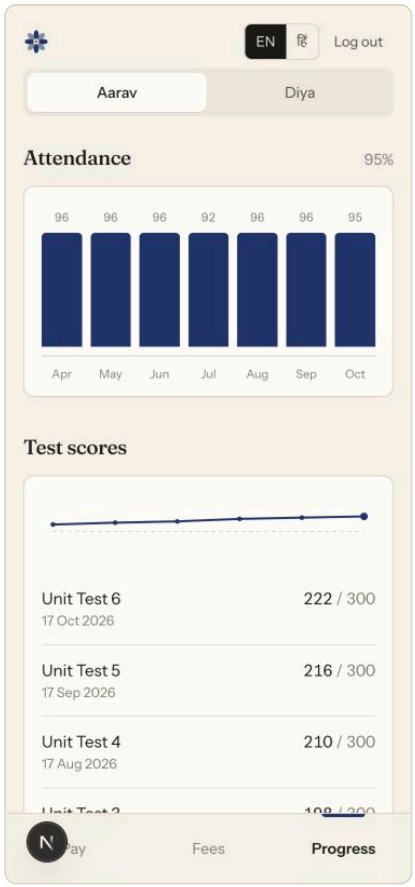
What the parent received. Staff can open the exact WhatsApp thread for any student, useful when a parent calls to dispute a message.



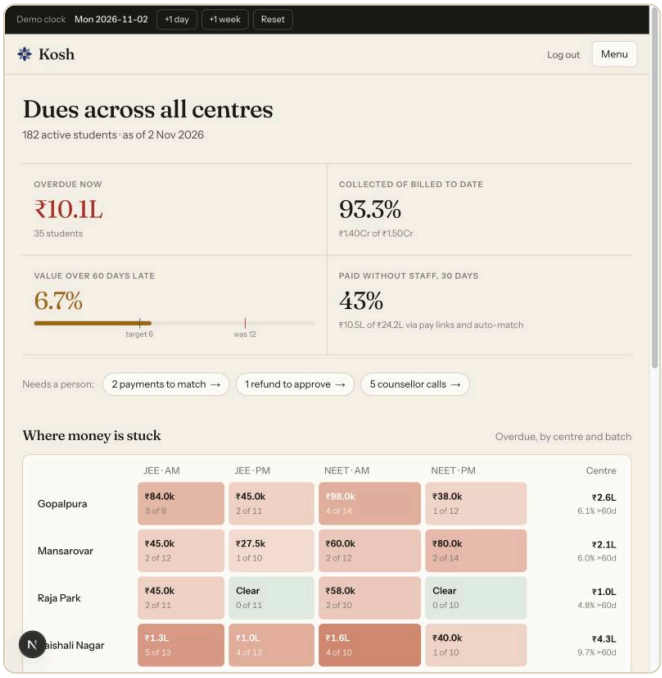
DPDP consent. Shown once, in the parent's language: what, why, how long, their rights. "Not now" still shows fees but hides attendance and scores.



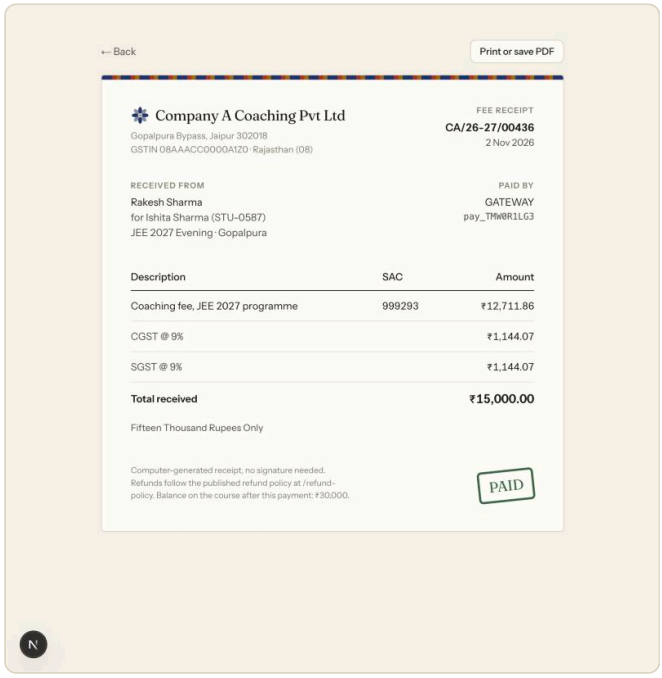
Fee plan. Balance against the net fee, each instalment with a status word, then receipts. Discounts are shown with their reason.



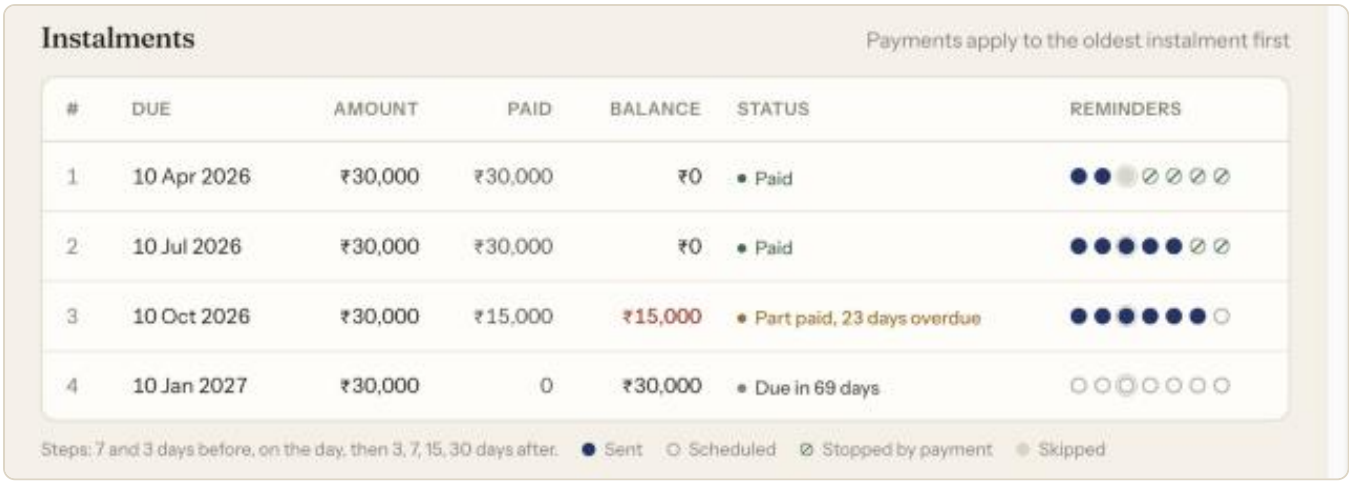
Progress. Monthly attendance as bars that turn ochre then madder below 85% and 75%, and test scores with a trend line. A switcher for families with more than one child.



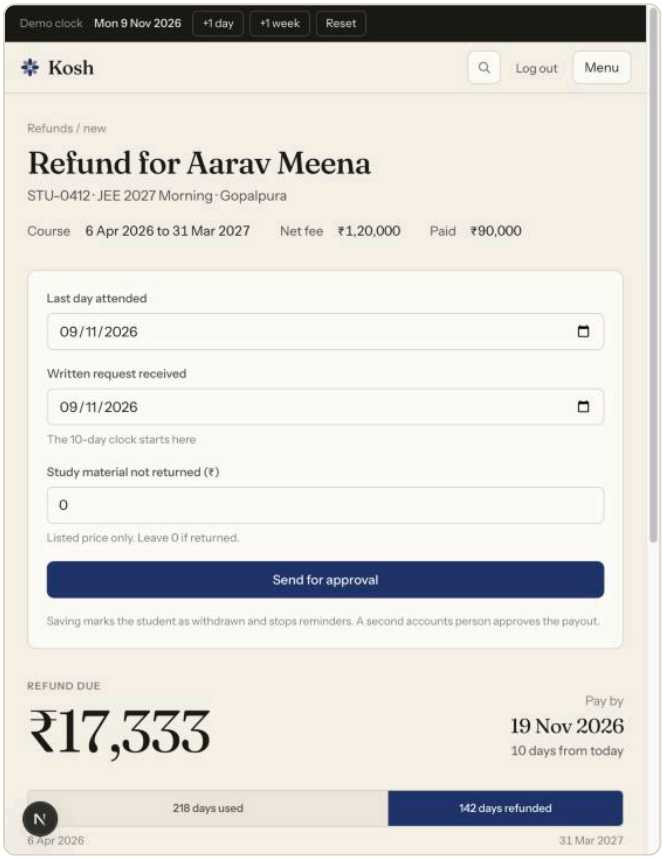
Dues desk. Headline figures separated by hairlines, the 60-day metric drawn against the brief's 12% baseline and 6% target, and a centre × batch heatmap that answers "where is money stuck" and links straight to the filtered list.



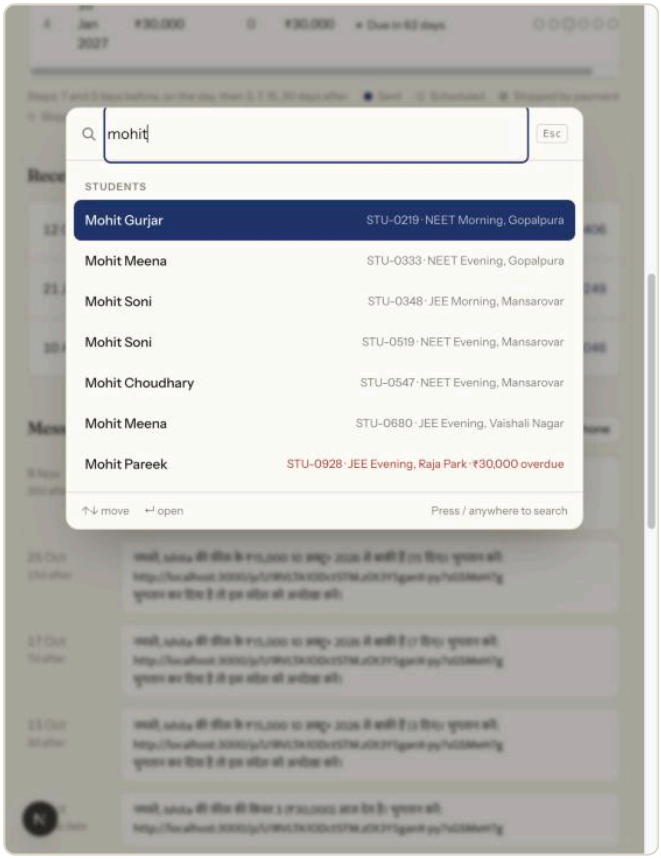
GST receipt. Taxable value, CGST and SGST, SAC, amount in words in lakh/crore. Printable or saved as PDF. Reachable by the parent through a signed link without logging in.



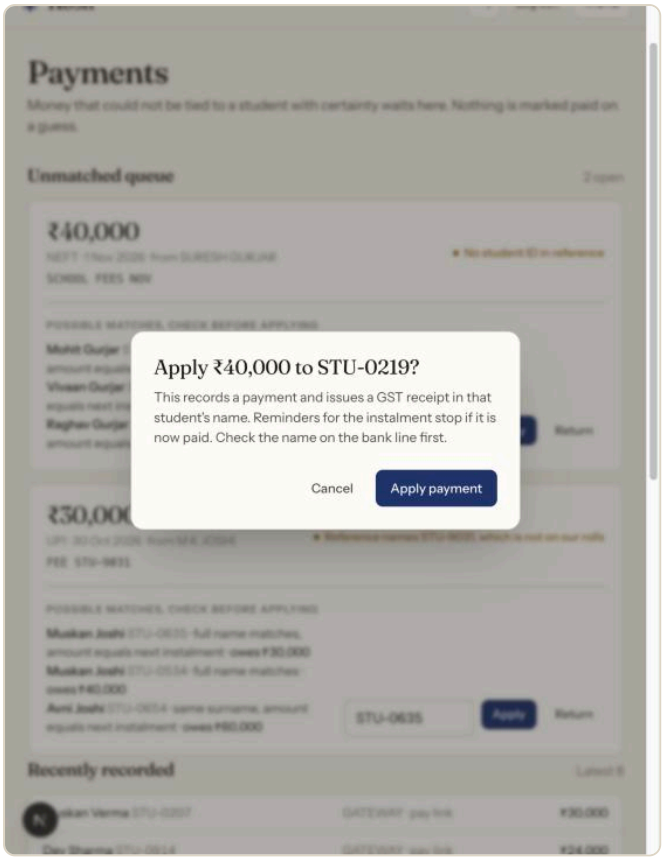
Reminder trail on the ledger. One dot per step: filled = sent, ring = scheduled, slashed = stopped by payment. This makes "reminders stop on payment" visible at a glance rather than something staff have to trust.



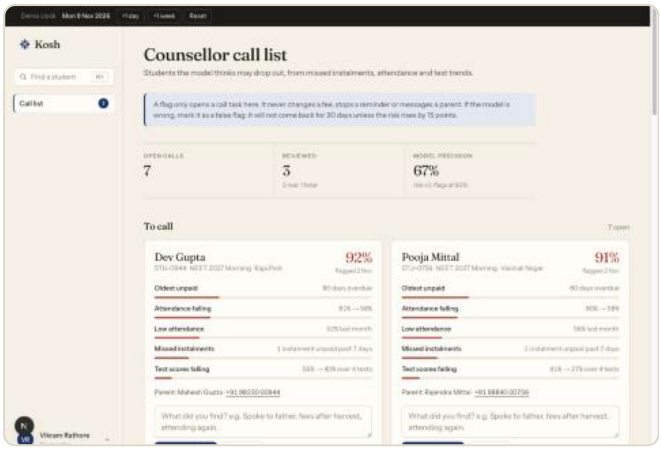
Refund calculator. Live as you type; the course drawn as used vs refunded days; each line tagged with its policy clause; the deadline as a date and a countdown.



Find anyone, fast. Press / or ⌘K anywhere on the desk. Overdue students show their amount in madder in the results.



Confirm before money moves. Applying a credit, returning one, approving or paying a refund, sending a refund for approval, marking a false flag, and resetting the demo all ask first, in plain words about the consequence.



Counsellor call list. Probability, the reasons as bars, the parent's number as a tap-to-call link, and a note box. The rules about what a flag can and cannot do are stated on the page.

States and details

- Loading skeletons on the desk, pending spinners on every submit button, empty states with a next step, a page-level error boundary that says nothing was changed, and a not-found page. Toasts confirm each action even when the form is off-screen.
- Forms keep what was typed when the server rejects them (React 19 clears forms by default; I submit through `onSubmit` to avoid that).
- Phone layouts: parent pages are designed for 360 px first; the student list becomes a card list below tablet width and wide tables scroll inside their card; the desk menu collapses.
- Accessible basics: labelled inputs, native `<dialog>` modals with focus handling and Esc, visible focus rings, reduced-motion respected, colour never the only signal (every status has a word).

---

8. Security and privacy

- **Sign-in.** Parents: 6-digit OTP, HMAC-hashed at rest, 5-minute expiry, 5 tries, 3 sends per 10 minutes, same reply for unknown numbers. Staff: bcrypt passwords, constant-time comparison even for unknown emails. Sessions are signed HS256 cookies, httpOnly, SameSite=Lax, Secure in production.
- **Access control.** Middleware gates `/parent` and `/staff`, and every page and server action checks the role again. Parents only ever load students whose parent ID equals their session ID. Counsellors see only the call list; only counsellors can close flags; only accounts can touch money or export.

- **Money integrity.** Amounts come from the server ledger, not the browser. Gateway callbacks are HMAC-verified, idempotent and re-checked inside a transaction. Receipt numbers are allocated inside the same transaction. Refunds are recomputed on the server and need a second person.
- **Inputs.** Zod validation on every action; CSV uploads are all-or-nothing with per-line errors; CSV exports guard against spreadsheet formula injection; XML is escaped.
- **DPDP.** Consent screen on first login explains what, why, how long and the parent's rights; consent is stored with version and time against the OTP-verified number and audited. Parents who skip it still see fees (needed to run the contract) but not attendance or scores.
- **Secrets.** None in the repo. `npm run setup` generates them locally. All personal data in the repo is fictional seed data.

---

9. Tests

62 Vitest tests over the rules that cost money if wrong: the three sample rows reproduce the brief's status text exactly; FIFO allocation, overpayment credit and lateness; instalment splits always summing to the net fee; every refund clause including late joiners, leaving before start, shortfall and GST credit notes; reminder timing, no resend, no burst, stop on payment, no em dashes in templates; risk bands including "unsure" and "too little data", false-flag suppression; matching with IDs written several ways, overpayment and name-only suggestions; GST paise rounding; amount in words; CSV parsing and validation; Tally escaping and formula injection; forged and expired sessions, edited pay links, receipt keys and phone formats. Every screen was also clicked through in a browser at phone and desktop widths, including the full pay-to-receipt path.

---

10. What I cut, and what I would do next

Cut or mocked, on purpose

- Real gateway, SMS/WhatsApp and bank feed. Each sits behind a seam that matches the real contract.
- Creating fee plans and students in the UI (they come from the existing fee software; seeded here).
- Student login. The brief's core roles are parent and accounts.
- Per-centre staff scoping; everyone on the desk sees all centres, which is what the founder asked for.
- Rate limiting for staff login beyond bcrypt cost.

Next, in order

1. Razorpay or Cashfree in place of the mock, using the same webhook handler; WhatsApp Business templates for the reminder text (approved templates need these exact words).
2. Bank statement import (CSV/MT940) feeding the matcher daily.
3. Two-way Tally sync and a month-end close checklist, to get reconciliation from 4 days to 1.
4. Refit the risk model on one term of counsellor outcomes and real withdrawals; report precision and recall per centre.
5. Consent withdrawal and data export for parents; per-centre staff permissions; staff login throttling and 2FA.
6. Measure the brief's targets: parent calls about dues (needs the call log), 60-day late value by term.

11. Limits, honestly

- Risk weights are judgement, not fitted. Until there is outcome data, precision on the page is the only evidence it works.
- Ledgers are computed in memory per request. That is fine for thousands of students; at tens of thousands I would cache per-student balances and invalidate on payment.
- The refund policy wording is mine. The client's lawyer should sign it off before it is published.
- The demo clock and on-screen OTP exist only in demo mode and must be off (`DEMO_MODE=false`) for a real launch.
- Dependencies are on patched releases (Next.js 15.5.28, drizzle-orm 0.45.4, zod 4.6.7). `npm audit` still lists postcss and sharp inside Next.js; the fix is Next.js 16. postcss only processes this project's own CSS at build time, and sharp (image resizing) is not used. I would move to Next.js 16 after submission rather than take a major upgrade under deadline.
- Hindi copy was written carefully but should be read by a native speaker at the client before parents see it.